Last time: Say W is a nontrivial subspace of some $\mathbb{R}^m$ with orthogonal basis $\{u_1, u_2, \ldots, u_n\}$

Naturally we must always have $n \leq m$ in this setting.

Then for any $y \in \mathbb{R}^m$, we have

$$\text{proj}_W y = \hat{y} = \sum_{i=1}^{n} \frac{y \cdot u_i}{u_i \cdot u_i} u_i$$

Showed that $y - \hat{y} \in W^{\perp}$ so that we can always decompose y into the sum of a vector in W and one in $W^{\perp}$.

The fact that such a breakdown always exists is called the <u>Orthogonal Decomposition Theorem</u>

But is this decomposition unique and how does it help? Will answer both those questions now.

One good use of projection is that $\hat{y}$ is the "best approximation" of y in W, said differently the point in W which is closest to y. That is, for any $w \in W$ we have

$$\| y - \hat{y} \| \leq \| y - w \|$$

with these being equal only if $w = \hat{y}$. So not only is $\hat{y}$ closest, it is unique in being that close.

Why? Note

$$y - w = \underbrace{(y - \hat{y})}_{\in W^{\perp}} + \underbrace{(\hat{y} - w)}_{\in W}$$

Remember other day we proved for u, v orthogonal,

$$\| u + v \|^2 = (u + v) \cdot (u + v) = \| u \|^2 + \| v \|^2$$

So in prior identity for $y - w$, dotting each side with itself and using this idea yields

$$\| y - w \|^2 = \| y - \hat{y} \|^2 + \| \hat{y} - w \|^2$$

This proves

$$\| y - \hat{y} \| \leq \| y - w \|$$

as desired with them only equal if $\hat{y} = w$ which is also as stated, proving that $\hat{y}$ is the unique closest point in W to y.

And this also shows the breakdown

$$y = \underbrace{\hat{y}}_{\in W} + \underbrace{z}_{\in W^{\perp}}$$

is unique as $\hat{y}$ is the unique point in W closest to y.

If there was another such breakdown we would similarly show the portion of that which is in W would also be the unique closest point to y, and of course that doesn't make sense to have two "unique" closest points. Hence why this breakdown is unique.

Computing projection can be long if subspace has reasonably high dimension even if we are already given an orthogonal basis. One way can make it notationally a bit more compact is as follows. Note that it's not that this approach makes it much easier, it doesn't, it's more that this idea can be expanded into broader circumstances.

Say the orthogonal basis we had for W was actually orthonormal, and we made those basis vectors into the columns of a matrix U (so U has orthonormal columns).

Then turns out

$$\text{proj}_W y = \hat{y} = \underbrace{UU^T}_{\text{not I!}} y$$

We have $U^T U = I$ not the reverse! Reverse will only be so if U is a square matrix.

And U being square wouldn't really happen here. If it was, since columns of U are a basis for W, that would mean W is the full space $\mathbb{R}^m$ so that $\text{proj}_W y = y$

And that's fine but obviously not interesting.

Why does that hold? Well note that (u_i's columns of U)

$$U^T y = \begin{bmatrix} y \cdot u_1 \\ y \cdot u_2 \\ \vdots \\ y \cdot u_n \end{bmatrix}$$

And then

$$UU^T y = \sum_{i=1}^{n} (y \cdot u_i) u_i$$

as needed since $u_i \cdot u_i = 1$ for all i since orthonormal basis so this then matches the projection formula.

Again, this formula doesn't make it much easier or anything compared to the original, just useful to look at things in different ways. And this shows yet another use of transpose, in particular how it interacts when multiplied by the original (will see more of this later!).

Plus if you wanted to find the projection for multiple points, rather than using the original formula from scratch, likely more convenient to use this as even though the y would change for each point, the UU^T part would stay the same, only have to do that once.

6.3.13

$$x = \begin{bmatrix} 3 \\ -7 \\ 2 \\ 3 \end{bmatrix} \quad ; \quad v_1 = \begin{bmatrix} 2 \\ -1 \\ -3 \\ 1 \end{bmatrix} \quad , \quad v_2 = \begin{bmatrix} 1 \\ 1 \\ 0 \\ -1 \end{bmatrix}$$

Find point in subspace spanned by $\{v_1, v_2\}$ which is closest to x.

Since v_1, v_2 are clearly linearly independent, they generate a plane, not an unfamiliar setting. So if this plane were in three dimensions we might use say cross product to do this. But since this is in four dimensions, what do we do?

Fortunately here we see that $v_1 \cdot v_2 = 0$ so the given basis is orthogonal. Thus just use projection formula:

$$\hat{x} = \frac{x \cdot v_1}{v_1 \cdot v_1} v_1 + \frac{x \cdot v_2}{v_2 \cdot v_2} v_2$$

$$= \frac{10}{15} \begin{bmatrix} 2 \\ -1 \\ -3 \\ 1 \end{bmatrix} + \frac{-7}{3} \begin{bmatrix} 1 \\ 1 \\ 0 \\ -1 \end{bmatrix}$$

$$= \frac{1}{3} \begin{bmatrix} 4 - 7 \\ -2 - 7 \\ -6 + 0 \\ 2 + 7 \end{bmatrix} = \begin{bmatrix} -1 \\ -3 \\ -2 \\ 3 \end{bmatrix}$$

But this raises immediate question of what we would do if basis was not orthogonal? Must be careful about that! Very common in this type of question you might be given it with a nonorthogonal basis.

And often, as mentioned, in that setting people apply the projection formula immediately still. And you must not do that - it will definitely not be correct!

So what is the right thing to do? Need to change the "ordinary" basis into an orthogonal one so we can then use that formula. But how? That's what next section is about.

6.4 Gram-Schmidt Process/QR Factorization

Say we have a subspace of $\mathbb{R}^m$ for which we perhaps have a basis but for whatever reason (projections perhaps) we need an orthogonal basis.

For $\mathbb{R}^m$ itself we can just use standard basis, one of several reasons we prefer it since it's inherently always an orthonormal basis. But some subspace of $\mathbb{R}^m$ there might not be an orthogonal basis that is naturally/readily available as the standard basis is for the full space.

For instance taking Col A, Row A, Nul A for some appropriately sized matrix A via REF, in the usual way, the basis we find typically won't be orthogonal. How do we make it so?

Or even if we are in full space $\mathbb{R}^m$ suppose we want our orthogonal basis to include a certain vector (maybe for diagonalization reasons). What to do then as standard basis probably won't help after such a requirement.

We can actually use this upcoming idea in other vector spaces too, ones which have an inner product defined on them. But ok that comes later.

Say $\{x_1, x_2, \dots, x_n\}$ is a basis for subspace W of $\mathbb{R}^m$. Want to create new basis $\{v_1, v_2, \dots, v_n\}$ for W which is an orthogonal basis. But how?

Start with

$$v_1 = x_1$$

Then take

$$v_2 = x_2 - \underbrace{\frac{x_2 \cdot v_1}{v_1 \cdot v_1} v_1}_{\text{proj}_{v_1} x_2}$$

By what we showed earlier with projections we know v_2 is orthogonal to v_1

And since $v_1 = x_1$ and v_2 is in terms of x_2, v_1,

it easily follows that $\text{span}\{v_1, v_2\} = \text{span}\{x_1, x_2\}$

Meaning so far we didn't change span, and the two "new" vectors are orthogonal as needed. After this, take

$$v_3 = x_3 - \underbrace{\frac{x_3 \cdot v_1}{v_1 \cdot v_1} v_1 - \frac{x_3 \cdot v_2}{v_2 \cdot v_2} v_2}_{\text{proj}_L x_3}$$

where L is subspace spanned by $\{v_1, v_2\}$. So by same logic, v_3 is orthogonal to both v_1 and v_2

So now $\{v_1, v_2, v_3\}$ is an orthogonal set and by same logic again as before, this will have same span as $\{x_1, x_2, x_3\}$

Continuing, last one will become

$$v_n = x_n - \sum_{i=1}^{n-1} \frac{x_n \cdot v_i}{v_i \cdot v_i} v_i$$

By what we were doing successively $\{v_1, v_2, \dots, v_n\}$ is an orthogonal basis for same subspace that is spanned by $\{x_1, x_2, \dots, x_n\}$, namely W.

Not only that, for any k in $[1, n]$, as we also showed $\{v_1, v_2, \dots, v_k\}$ has same span as $\{x_1, x_2, \dots, x_k\}$

Useful in case you want to cut this off at some point, knowing that you haven't changed the span at that stage.

And fact that $v_1 = x_1$ if there is some vector we really want to keep from original basis, taking it to be the first vector is a way to do that.

But this brings up a more relevant issue. That is, what "order" should be used for $\{x_1, x_2, \ldots, x_n\}$

Of course the subscript labels might suggest something as maybe we should use the order they are listed in. But unless expressly told to (which would be unlikely) you definitely don't have to maintain that order. In fact there might be good reasons not to, depending on the situation.

For one, as stated if we really wanted to keep a certain vector in the set. But more importantly to perhaps make the calculations easier.

That's an unfortunate thing about Gram-Schmidt. Like many algorithms in this class, it's very important but in the situations we might really need it (higher dimensional subspaces) it's a lot of computation to do by hand.

As seen though, in the computation we do alot more computations with x_1 (that is, v_1) than say with x_{n-1} So if original basis had a vector with many 0s, it would likely be easier if we took that to be x_1 instead of x_{n-1}.

Or if you say noticed two of the vectors were already orthogonal to each other, taking those as x_1, x_2 would mean we'd have to do Gram-Schmidt one fewer time on whole as those are immediately v_1, v_2 .

These sorts of things can make it easier on us.

Of course switching order might not give us same orthogonal basis as another order, but that shouldn't matter usually, we typically just want an orthogonal basis, we don't care which one.

Note also as usual, once we have orthogonal basis is easy to make orthonormal - just divide each vector by its length as we typically do.

6.4.5

$$x_1 = \begin{bmatrix} 1 \\ -4 \\ 0 \\ 1 \end{bmatrix}, \quad x_2 = \begin{bmatrix} 7 \\ -7 \\ -4 \\ 1 \end{bmatrix}$$

not orthogonal

Want to create orthogonal basis for same subspace spanned by these two. Take

$$v_1 = x_1$$

Then

$$v_2 = x_2 - \frac{x_2 \cdot v_1}{v_1 \cdot v_1} v_1$$

$$= \begin{bmatrix} 7 \\ -7 \\ -4 \\ 1 \end{bmatrix} - \frac{36}{18} \begin{bmatrix} 1 \\ -4 \\ 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 5 \\ 1 \\ -4 \\ -1 \end{bmatrix}$$

Thus

$$\left\{ \begin{bmatrix} 1 \\ -4 \\ 0 \\ 1 \end{bmatrix}, \begin{bmatrix} 5 \\ 1 \\ -4 \\ -1 \end{bmatrix} \right\}$$

is the desired orthogonal basis.

Calculation not so bad here since only had to apply Gram-Schmidt once, and even within that, the dot product portion simplified cleanly so our final answer had two clean vectors.

What if that fraction didn't simplify nicely? Well by itself, that's not a big deal since you are done with this once v_2 is computed, i.e. no further calculations to be done after with that messy new vector.

But what if we needed say to go up to v_4 instead of just v_2 ? Then arriving at messy fraction the first time, say denominator had been 17 instead of 18, would make later steps much more irritating. And even if first fraction happens to be clean, almost certainly in some follow up step it won't stay clean, not realistic to think it will stay that way.

You could multiply new vector by an appropriate integer to clear out the fractions, but then you will almost certainly have some large integers to deal with, that isn't necessarily going to be an improvement!

Gram-Schmidt can be ugly, even for relatively small dimensional spaces, no way around that, is what it is.

QR Factorization

General context here is breaking a matrix down into a product of much simpler matrices to hopefully make make it easier to work with. That was one point of diagonalization.

But of course that only worked for certain square matrices. How might we break down some nonsquare matrix to make it easier to work with?

QR Factorization can help somewhat with that. But it and diagonalization are in a sense toned down versions of the generalized breakdown that will culminate the linear algebra portion of the class in 7.4 as it too, like diagonalization, doesn't apply to every real matrix.

<u>QR Factorization:</u> Say A is a real $m \times n$ matrix with linearly independent columns. First off, this inherently requires $m \geq n$ for otherwise the columns could not be linearly independent.

So although this can be applied to some nonsquare matrices, it's still restricted to only matrices which have linearly independent columns (which as mentioned must have at least as many rows as columns) so this is a fairly big restriction.

Want to write A in form

$$A = QR$$

where Q is an $m \times n$ matrix whose column space is the same as A's, just Q having orthonormal columns. And R is an $n \times n$ upper triangular matrix with all diagonal entries positive.

Point being that Q, R are much simpler than A usually so hopefully this helps.

R having positive entries on diagonal isn't that important, negative or mix wouldn't be much issue. More that we can make it such so why not to generally make it easier and more consistent.

Naturally easier to give this type of instruction to a machine too, it's more concrete.

But this does inherently say R is invertible because it can't have a 0 on the diagonal and being triangular so that means it has nonzero determinant.

Why is that? Well A is $m \times n$, with m at least as large as n and having linearly independent columns which ensures rank $A = n$. And Q, R both have n columns so each has rank at most n. However rank of product is $\leq$ rank of each (proved in worksheet), so rank Q = rank $R = n$ is forced.

So since R is $n \times n$, that ensures R is invertible.

But how do we find Q, R? Since Q's columns are to have same span as A's, and A's columns are linearly independent, we use Gram-Schmidt on the columns of A to create an orthogonal basis for Col A which is what is needed (can then turn that orthonormal easily).

That's all for Q. Creating R, bit more to it, including making all diagonal entries positive. Will discuss that next time.